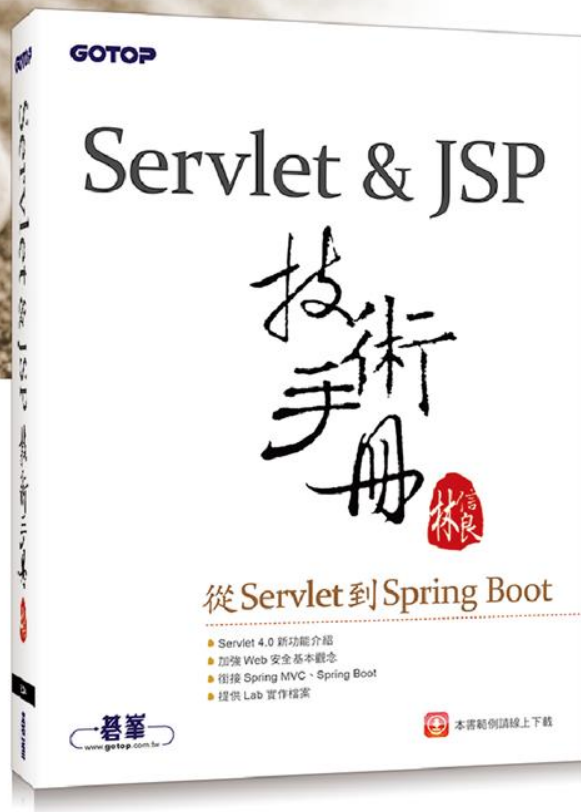




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

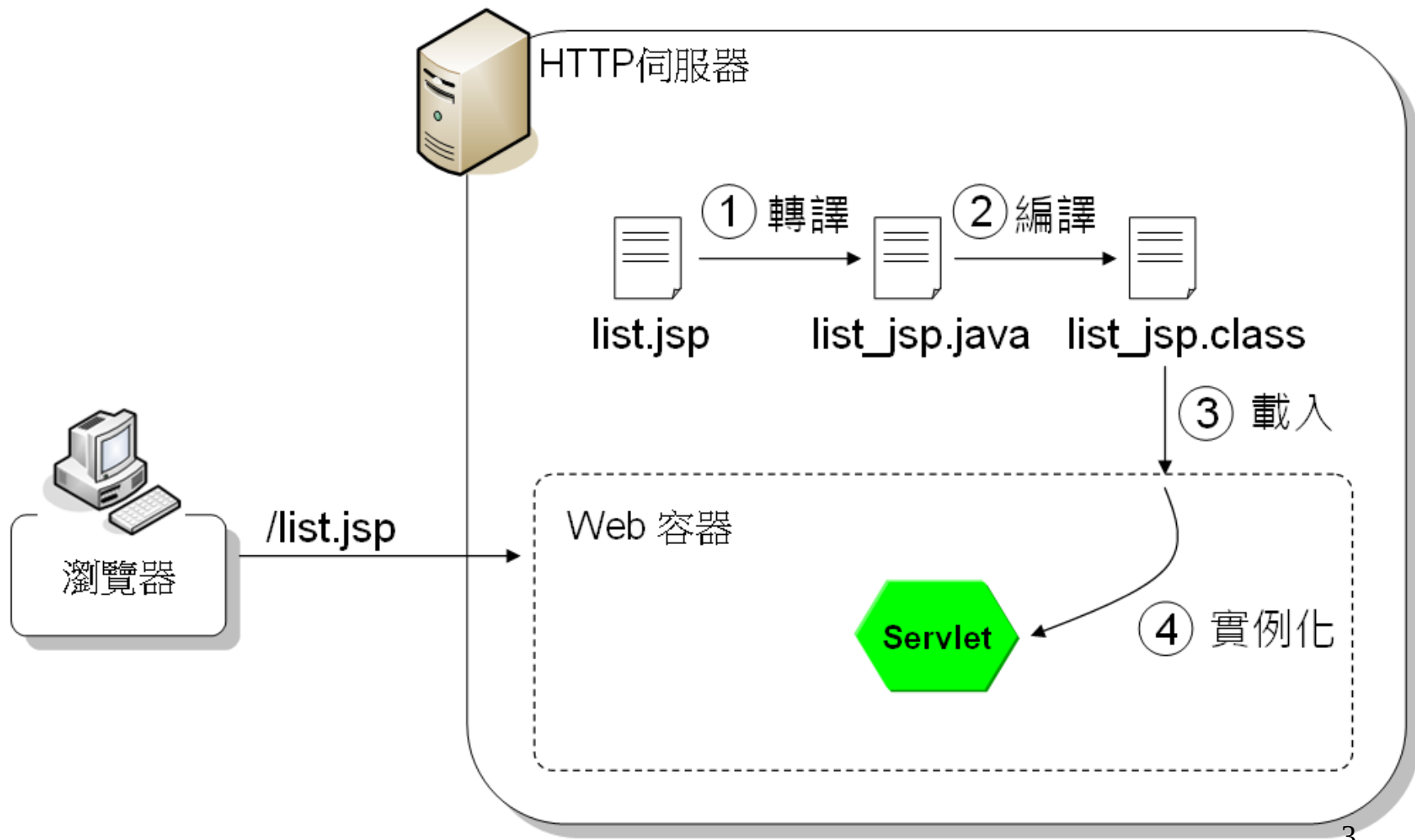
- 使用JSP



學習目標

- 了解JSP生命週期
- 使用JSP語法元素
- 使用JSP標準標籤
- 了解何謂Model 1架構
- 使用運算式語言（EL）
- 自訂EL函式

JSP生命週期



```
<%@page import="java.time.LocalDateTime"%>
<%@page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>JSP 範例文件</title>
  </head>
  <body>
    <!-- 這邊會依 Web 網站的時間而產生不同的回應 -->
    <%= LocalDateTime.now() %>
  </body>
</html>
```

```
package org.apache.jsp;

import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.jsp.*;
import java.time.LocalDateTime;

public final class time_jsp extends org.apache.jasper.runtime.HttpJspBase
    implements org.apache.jasper.runtime.JspSourceDependent,
               org.apache.jasper.runtime.JspSourceImports {

    // 略...

    public void _jspInit() {
    }

    public void _jspDestroy() {
    }

    public void _jspService(final javax.servlet.http.HttpServletRequest request,
        final javax.servlet.http.HttpServletResponse response)
        throws java.io.IOException, javax.servlet.ServletException {

        final java.lang.String _jspx_method = request.getMethod();
        if (!"GET".equals(_jspx_method) && !"POST".equals(_jspx_method)
        && !"HEAD".equals(_jspx_method)
        && !javax.servlet.DispatcherType.ERROR.equals(request.getDispatcherType())) {
            response.sendError(HttpServletResponse.SC_METHOD_NOT_ALLOWED, "JSPs only
            permit GET POST or HEAD");
            return;
        }
        // 略...
```

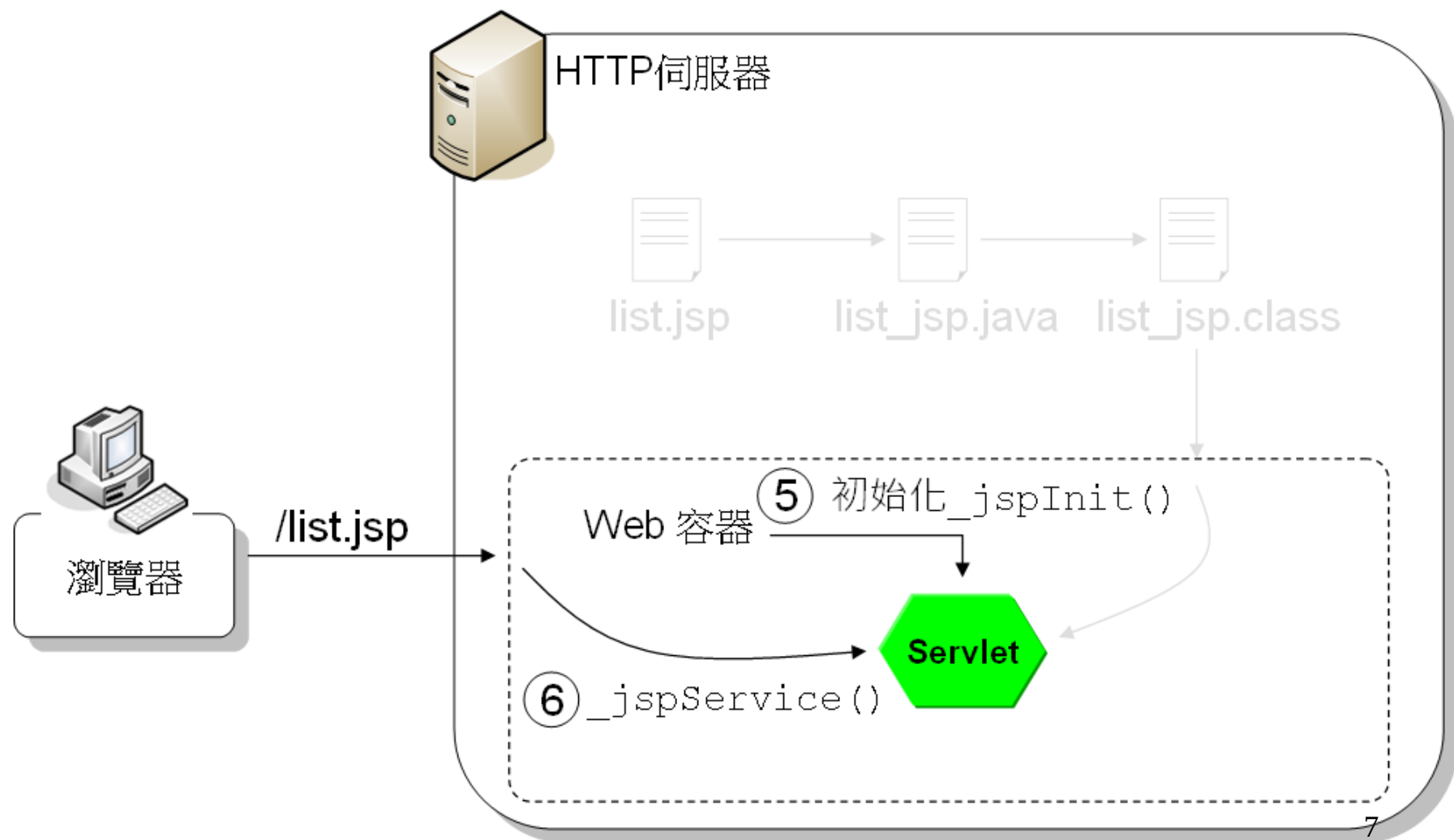
從Java EE 7的JSP 2.3開始
JSP只接受GET、POST、HEAD請求

```
try {
    response.setContentType("text/html; charset=UTF-8");
    pageContext = _jspxFactory.getPageContext(this, request, response,
                                              null, true, 8192, true);
    // 略...

    out = pageContext.getOut();
    _jspx_out = out;

    out.write("\r\n");
    out.write("\r\n");
    out.write("<!Doctype html>\r\n");
    out.write("<html>\r\n");
    out.write("    <head>\r\n");
    out.write("        <meta charset=\"UTF-8\">\r\n");
    out.write("        <title>JSP 範例文件</title>\r\n");
    out.write("    </head>\r\n");
    out.write("    <body>\r\n");
    out.write("        ");
    out.print( LocalDateTime.now() );
    out.write("\r\n");
    out.write("    </body>\r\n");
    out.write("</html>");
} catch (java.lang.Throwable t) {
    // 略...
} finally {
    _jspxFactory.releasePageContext(_jspx_page_context);
}
}
```

JSP生命週期



```
package org.apache.jasper.runtime;
// 略...

public abstract class HttpJspBase extends HttpServlet
                                implements HttpJspPage {
    // 略...
    public final void init(ServletConfig config)
                                throws ServletException {
        super.init(config);
        jspInit();
        _jspInit();
    }
    // 略...
    public final void destroy() {
        jspDestroy();
        _jspDestroy();
    }
    public final void service(HttpServletRequest request,
                                HttpServletResponse response)
                                throws ServletException, IOException {
        _jspService(request, response);
    }
    public void jspInit() {    }
    public void _jspInit() {    }
    public void jspDestroy() {    }
    protected void _jspDestroy() {    }
```


Servlet至JSP的簡單轉換

```
response.setContentType("text/html;charset=UTF-8");
PrintWriter out = response.getWriter();
out.println("<!DOCTYPE html>");
out.println("<html>");
out.println("<head>");
out.println("<meta charset='UTF-8'>");
out.println("<title>觀看線上書籤</title>");
out.println("</head>");
out.println("<body>");
out.println("<table style='text-align: left; width: 100%;' border='0' >");
out.println("  <tbody>");
out.println("    <tr>");
out.println("      <td style='background-color: rgb(51, 255, 255); '>網頁</td>");
out.println("      <td style='background-color: rgb(51, 255, 255); '>分類</td>");
```

Servlet至JSP的簡單轉換

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>  
PrintWriter out = response.getWriter();  
out.println("<!DOCTYPE html>");  
out.println("<html>");  
out.println("<head>");  
out.println("<meta charset='UTF-8'>");  
out.println("<title>觀看線上書籤</title>");  
out.println("</head>");  
out.println("<body>");  
out.println("<table style='text-align: left; width: 100%;' border='0' >");  
out.println("  <tbody>");  
out.println("    <tr>");  
out.println(  
    "    <td style='background-color: rgb(51, 255, 255);' >網頁</td>");  
out.println(  
    "    <td style='background-color: rgb(51, 255, 255);' >分類</td>");
```

Servlet至JSP的簡單轉換

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html>
    <head>
        <meta charset='UTF-8'>
        <title>觀看線上書籤</title>
    </head>
    <body>
        <table style='text-align: left; width: 100%;' border='0'>
            <tbody>
                <tr>
                    <td style='background-color: rgb(51, 255, 255); '>網頁</td>
                    <td style='background-color: rgb(51, 255, 255); '>分類</td>
```

```
BookmarkService bookmarkService = (BookmarkService)
    getServletContext().getAttribute("bookmarkService");
for(Bookmark bookmark : bookmarkService.getBookmarks()) {
    out.println("    <tr>");
    out.println("        <td><a href='http://' + bookmark.getUrl() +
        "'">" + bookmark.getTitle() + "</a></td>");
    out.println("        <td>" + bookmark.getCategory() + "</td>");
    out.println("    </tr>");
}
```

```
<%  
    BookmarkService bookmarkService =  
        (BookmarkService) application.getAttribute("bookmarkService");  
    for(Bookmark bookmark : bookmarkService.getBookmarks()) {  
%>  
        out.println("    <tr>");  
        out.println("        <td><a href='http://' + bookmark.getUrl() +  
            "'>" + bookmark.getTitle() + "</a></td>");  
        out.println("        <td>" + bookmark.getCategory() + "</td>");  
        out.println("    </tr>");  
    }  
}
```

```
<%
BookmarkService bookmarkService =
    (BookmarkService) application.getAttribute("bookmarkService");
for(Bookmark bookmark : bookmarkService.getBookmarks()) {
%>
    <tr>
        <td><a href='http://<%= bookmark.getUrl() %>'>
            <%= bookmark.getTitle() %></a></td>
        <td><%= bookmark.getCategory() %></td>
    </tr>
}
```

```
<%  
    BookmarkService bookmarkService =  
        (BookmarkService) application.getAttribute("bookmarkService");  
    for(Bookmark bookmark : bookmarkService.getBookmarks()) {  
%>  
        <tr>  
            <td><a href='http://<%= bookmark.getUrl() %>'>  
                <%= bookmark.getTitle() %></a></td>  
            <td><%= bookmark.getCategory() %></td>  
        </tr>  
%>  
    }  
%>
```

```
<%  
    BookmarkService bookmarkService =  
        (BookmarkService) application.getAttribute("bookmarkService");  
    for(Bookmark bookmark : bookmarkService.getBookmarks()) {  
%>  
        <tr>  
            <td><a href="http://<%= bookmark.getUrl()%>">  
                <%= bookmark.getTitle()%></a></td>  
            <td><%= bookmark.getCategory()%></td>  
        </tr>  
%>  
    }  
%>  
        </tbody> <tbody>");  
        </table> </table>");  
        </body> </body>");  
</html> </html>");  
    out.close();
```



```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cc.openhome.model.*, java.util.*" %>
<!DOCTYPE html>
<html>
    <head>
        <meta charset='UTF-8'>
        <title>觀看線上書籤</title>
    </head>
    <body>
        <table style='text-align: left; width: 100%;' border='0'>
            <tbody>
                <tr>
                    <td style='background-color: rgb(51, 255, 255);'>網頁</td>
                    <td style='background-color: rgb(51, 255, 255);'>分類</td>
                </tr>
            <%
                BookmarkService bookmarkService =
                    (BookmarkService) application.getAttribute("bookmarkService");
                for(Bookmark bookmark : bookmarkService.getBookmarks()) {
            %>
                <tr>
                    <td><a href='http://<%= bookmark.getUrl()%>'>
                        <%= bookmark.getTitle()%></a></td>
                    <td><%= bookmark.getCategory()%></td>
                </tr>
            <%
                }
            %>
            </tbody>
        </table>
    </body>
</html>
```


指示元素

`<%@ 指示類型 [屬性="值"] * %>`

- **page**
- **include**
- **taglib**

page指示元素

- 指示如何轉譯...

```
import java.util.Date;
```

```
<%@page import="java.util.Date" %>
```

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```
response.setContentType("text/html; charset=UTF8");
```

page指示元素

- 可用屬性
 - info
 - autoFlush
 - buffer
 - errorPage
 - extends
 - isErrorPage
 - language
 - session
 - isELIgnored
 - isThreadSafe

include指示元素

```
<%@ page pageEncoding="UTF-8" %>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
    <head><title>include 示範開頭</title></head>
    <body>
```

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@include file="/WEB-INF/jspf/header.jspf"%>
    <h1>include 示範本體</h1>
<%@include file="/WEB-INF/jspf/foot.jspf"%>
```

```
<%@ page pageEncoding="UTF-8" %>
    </body>
</html>
```

include指示元素

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
  <head><title>include 示範開頭</title></head>
  <body>
    <h1>include 示範本體</h1>
  </body>
</html>
```

指示元素

- 可以在web.xml中統一預設的網頁編碼、內容類型、緩衝區大小等

```
<web-app ...>
...
<jsp-config>
  <jsp-property-group>
    <url-pattern>*.jsp</url-pattern>
    <page-encoding>UTF-8</page-encoding>
    <default-content-type>text/html</default-content-type>
    <buffer>16kb</buffer>
  </jsp-property-group>
</jsp-config>
</web-app>
```


指示元素

- 可以宣告指定的JSP開頭與結尾要包括的網頁

```
<web-app ...>
  ...
  <jsp-config>
    <jsp-property-group>
      <url-pattern>*.jsp</url-pattern>
      <include-prelude>/WEB-INF/jspf/pre.jspf</include-prelude>
      <include-coda>/WEB-INF/jspf/coda.jspf</include-coda>
    </jsp-property-group>
  </jsp-config>
</web-app>
```

宣告元素

- 轉譯為Servlet中的類別成員或方法

<%! 類別成員宣告或方法宣告 %>

<%!

```
String name = "caterpillar";
```

```
String password = "123456";
```

```
boolean checkUser(String name, String password) {
```

```
    return this.name.equals(name) &&
```

```
        this.password.equals(password);
```

```
}
```

%>

宣告元素

```
package org.apache.jsp;
// 略...
public final class index_jsp
    extends org.apache.jasper.runtime.HttpJspBase
    implements org.apache.jasper.runtime.JspSourceDependent {
    String name = "caterpillar";
    String password = "123456";

    boolean checkUser(String name, String password) {
        return this.name.equals(name) &&
            this.password.equals(password) ;
    }
    // 略...
}
```

宣告元素

- 重新定義jspInit()、jspDestroy()

```
<%!  
    public void jspInit() {  
        // 初始化動作  
    }  
    public void jspDestroy() {  
        // 結尾動作  
    }  
%>
```

Scriptlet元素

- 轉譯為Servlet原始碼_jspService()方法中的內容

```
<% Java 陳述句 %>
```

```
<%
```

```
    String name = request.getParameter("name");  
    String password = request.getParameter("password");  
    if(checkUser(name, password)) {
```

```
%>
```

```
    <h1>登入成功</h1>
```

```
<%
```

```
    }  
    else {
```

```
%>
```

```
    <h1>登入失敗</h1>
```

```
<%
```

```
    }
```

```
%>
```

```
public final class login_jsp
    extends org.apache.jasper.runtime.HttpJspBase
    implements org.apache.jasper.runtime.JspSourceDependent {
    // 略...

    public void _jspService(HttpServletRequest request,
                                HttpServletResponse response)
        throws java.io.IOException, ServletException {
        // 略...

        String name = request.getParameter("name");
        String password = request.getParameter("password");
        if(checkUser(name, password)) {
            out.write("\n");
            out.write("    <h1>登入成功</h1>\n");
        }
        else {
            out.write("\n");
            out.write("    <h1>登入失敗</h1>\n");
        }
        // 略...
    }
}
```

運算式元素

- 運算結果將直接輸出為網頁的一部份

```
<%= Java 運算式 %>
```

```
<%= new Date() %>
```

```
out.print(new Date());
```

```
<%!  
    String name = "caterpillar";  
    String password = "123456";  
  
    boolean checkUser(String name, String password) {  
        return this.name.equals(name) &&  
            this.password.equals(password);  
    }  
%>  
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"  
    "http://www.w3.org/TR/html4/loose.dtd">  
<html>  
    <head>  
        <meta http-equiv="Content-Type"  
            content="text/html; charset=UTF-8">  
        <title>登入頁面</title>  
    </head>  
    <body>  
  
    <%  
        String name = request.getParameter("name");  
        String password = request.getParameter("password");  
        if(checkUser(name, password)) {  
    %>  
        <h1><%= name %> 登入成功</h1>  
    <%  
        } else {  
    %>  
        <h1>登入失敗</h1>  
    <%  
        }  
    %>  
    %>
```

使用宣告元素宣
告類別成員

使用運算式元素
輸出運算結果

使用 Scriptlet 撰寫
Java 程式片段

禁用Scriptlet

- 如果想禁用JSP上的Scriptlet，則可以在web.xml中設定

```
<web-app ...>
    ...
    <jsp-config>
        <jsp-property-group>
            <url-pattern>*.jsp</url-pattern>
            <scripting-invalid>true</scripting-invalid>
        </jsp-property-group>
    </jsp-config>
</web-app>
```

註解元素

```
<%  
    // 單行註解  
    out.println("隨便顯示一段文字");  
    /* 多行註解 */  
%>
```

<!-- 網頁註解 --> → out.write("<!-- 網頁註解 -->");

```
<%-- JSP 註解 -->
```

隱含物件

- 像out、request這樣的字眼，在轉譯為Servlet之後，會直接對應於_jspService()中的某個區域變數

隱含物件	說明
out	轉譯後對應 JspWriter 物件，其內部關聯一個 PrintWriter 物件。
request	轉譯後對應 HttpServletRequest 物件。
response	轉譯後對應 HttpServletResponse 物件。
config	轉譯後對應 ServletConfig 物件。
application	轉譯後對應 ServletContext 物件。
session	轉譯後對應 HttpSession 物件。
pageContext	轉譯後對應 PageContext 物件，它提供了 JSP 頁面資源的封裝，並可設定頁面範圍屬性。
exception	轉譯後對應 Throwable 物件，代表由其它 JSP 頁面丟出的例外物件，只會出現於 JSP 錯誤頁面（isErrorPage 設定為 true 的 JSP 頁面）。
page	轉譯後對應 this。

隱含物件

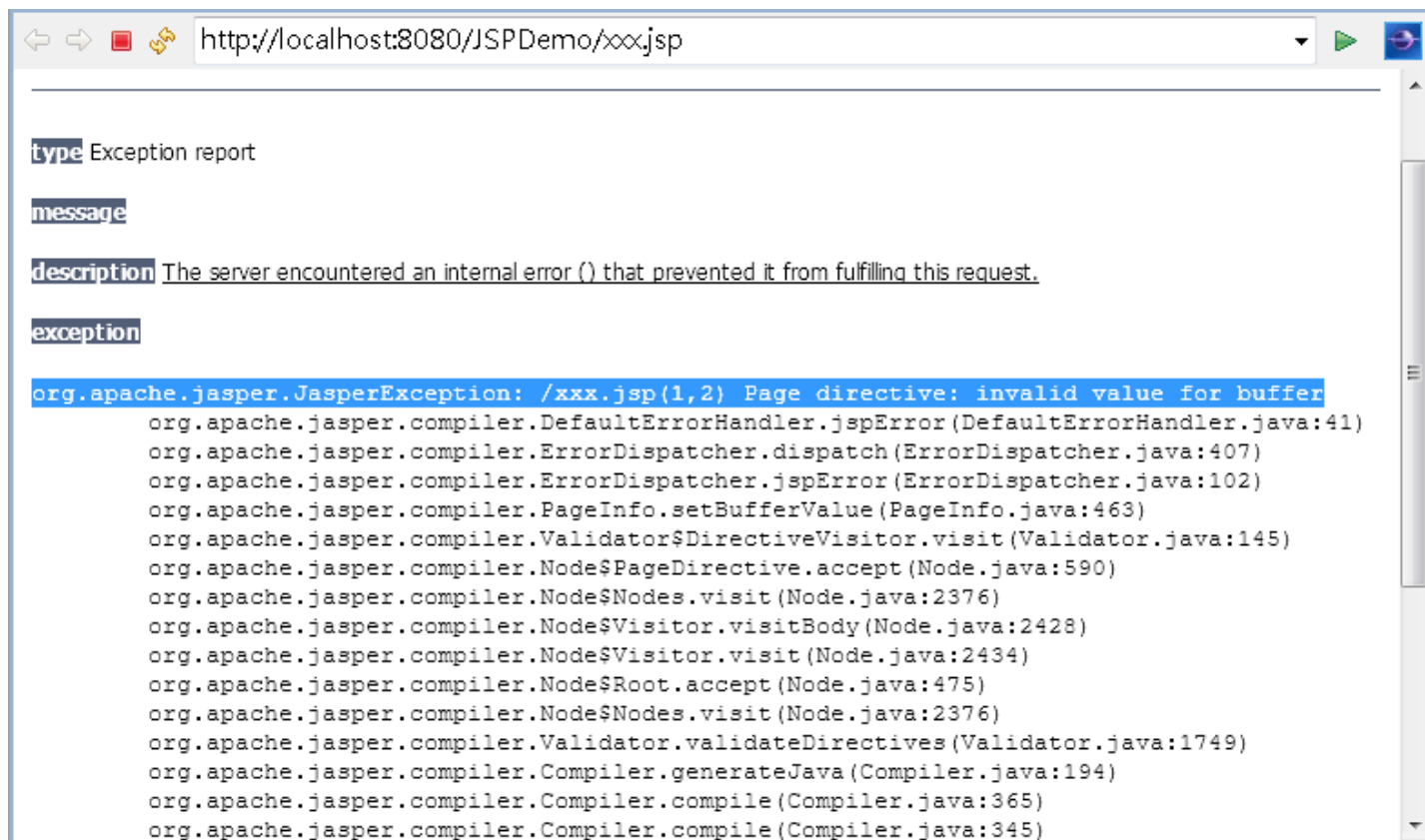
- 隱含物件只能在`<%與%>`之間，或`<%=與%>`之間使用
- 無法在`<%!與%>`之間使用隱含物件

錯誤處理

- JSP終究會轉譯為Servlet，所以錯誤可能發生在三個時候
 - JSP轉換為Servlet原始碼時
 - Servlet原始碼進行編譯時
 - Servlet載入容器進行服務但發生執行時期錯誤時

JSP轉換為Servlet原始碼時

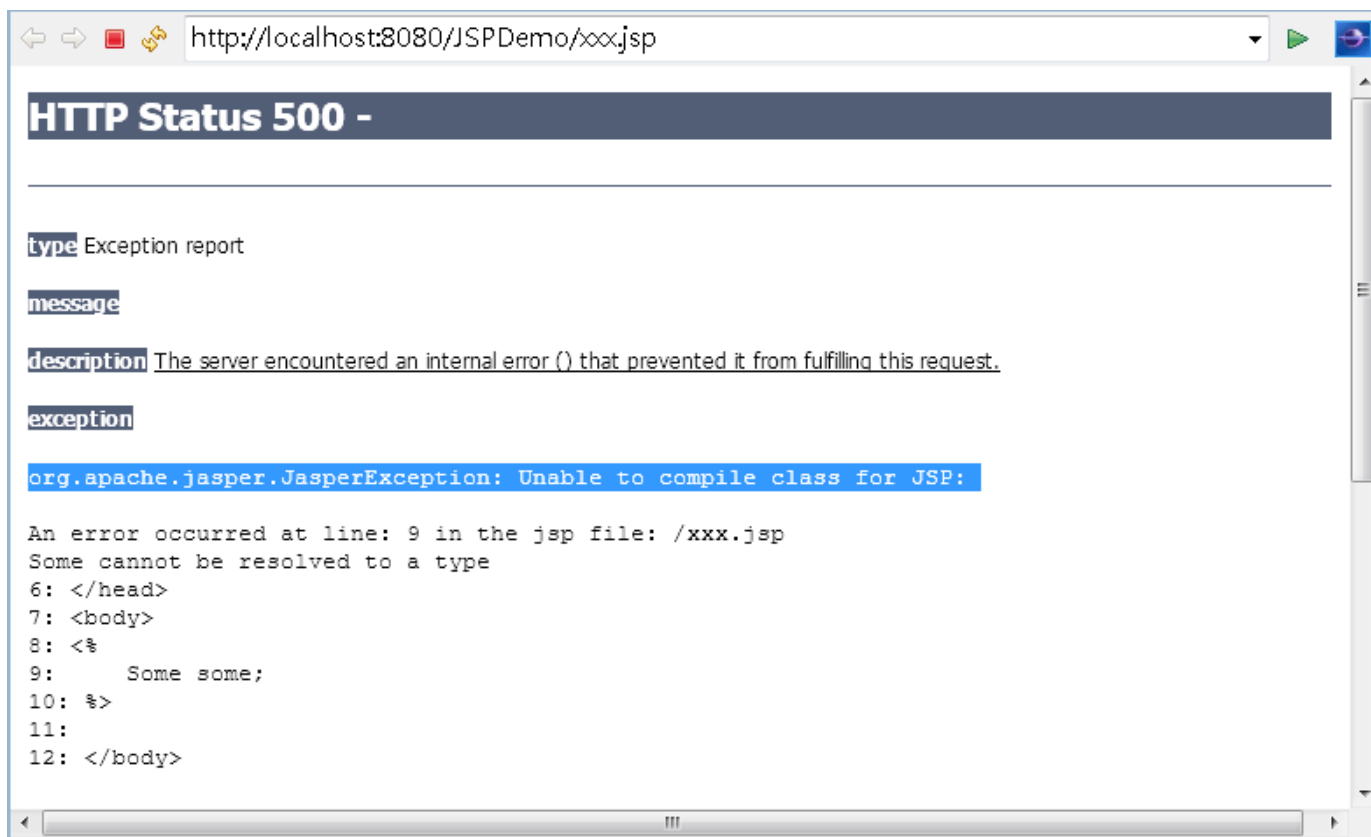
- 容器在轉譯JSP時不知道該怎麼將那些語法轉譯為Servlet的.java檔案



```
type Exception report
message
description The server encountered an internal error () that prevented it from fulfilling this request.
exception
org.apache.jasper.JasperException: /xxx.jsp(1,2) Page directive: invalid value for buffer
    org.apache.jasper.compiler.DefaultErrorHandler.jspError(DefaultErrorHandler.java:41)
    org.apache.jasper.compiler.ErrorDispatcher.dispatch(ErrorDispatcher.java:407)
    org.apache.jasper.compiler.ErrorDispatcher.jspError(ErrorDispatcher.java:102)
    org.apache.jasper.compiler.PageInfo.setBufferValue(PageInfo.java:463)
    org.apache.jasper.compiler.Validator$DirectiveVisitor.visit(Validator.java:145)
    org.apache.jasper.compiler.Node$PageDirective.accept(Node.java:590)
    org.apache.jasper.compiler.Node$Nodes.visit(Node.java:2376)
    org.apache.jasper.compiler.Node$Visitor.visitBody(Node.java:2428)
    org.apache.jasper.compiler.Node$Visitor.visit(Node.java:2434)
    org.apache.jasper.compiler.Node$Root.accept(Node.java:475)
    org.apache.jasper.compiler.Node$Nodes.visit(Node.java:2376)
    org.apache.jasper.compiler.Validator.validateDirectives(Validator.java:1749)
    org.apache.jasper.compiler.Compiler.generateJava(Compiler.java:194)
    org.apache.jasper.compiler.Compiler.compile(Compiler.java:365)
    org.apache.jasper.compiler.Compiler.compile(Compiler.java:345)
```

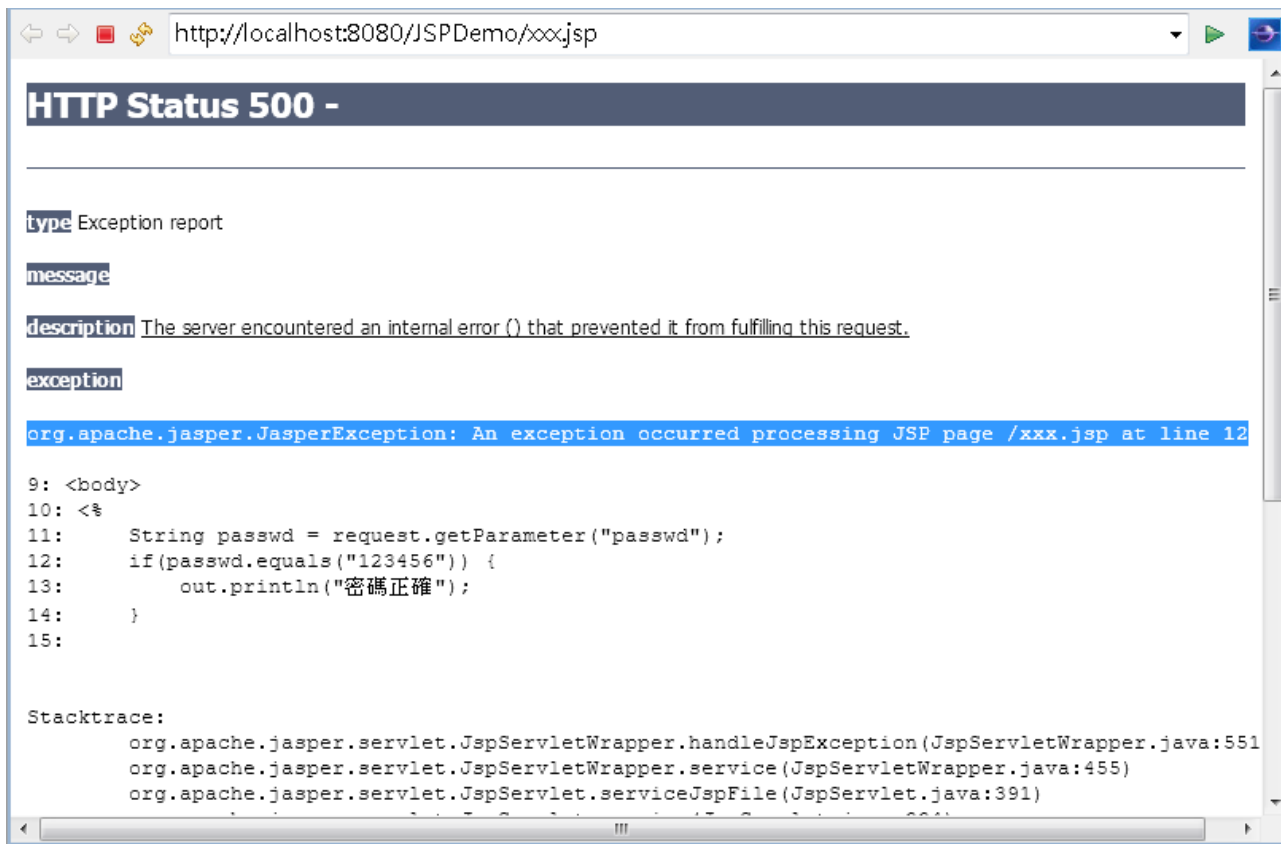
Servlet原始碼進行編譯時

- 嘗試將.java編譯為.class檔案，如果此時編譯器因為某個原因而無法完成編譯



進行服務但發生執行時期錯誤時

- 在執行時期因找不到某個資源、程式邏輯上的問題而發生錯誤



```
HTTP Status 500 -

type Exception report
message
description The server encountered an internal error () that prevented it from fulfilling this request.
exception
org.apache.jasper.JasperException: An exception occurred processing JSP page /xxx.jsp at line 12

9: <body>
10: <%
11:     String passwd = request.getParameter("passwd");
12:     if(passwd.equals("123456")) {
13:         out.println("密碼正確");
14:     }
15:

Stacktrace:
    org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:551)
    org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:455)
    org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:391)
```

進行服務但發生執行時期錯誤時

Tomcat v7.0 Server at localhost [Apache Tomcat] C:\Program Files\Java\jdk1.6.0\bin\jav

```
9: <body>
10: <%
11:     String passwd = request.getParameter("passwd");
12:     if(passwd.equals("123456")) {
13:         out.println("密碼正確");
14:     }
15:
```

Stacktrace:] with root cause

java.lang.NullPointerException

```
at org.apache.jsp.xxx_jsp._jspService(xxx_jsp.java:64)
at org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:71)
at javax.servlet.http.HttpServlet.service(HttpServlet.java:722)
at org.apache.jasper.servlet.JspServletWrapper.service(JspServletWr:
at org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.j:
at org.apache.jasper.servlet.JspServlet.service(JspServlet.java:334)
at javax.servlet.http.HttpServlet.service(HttpServlet.java:722)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter
```

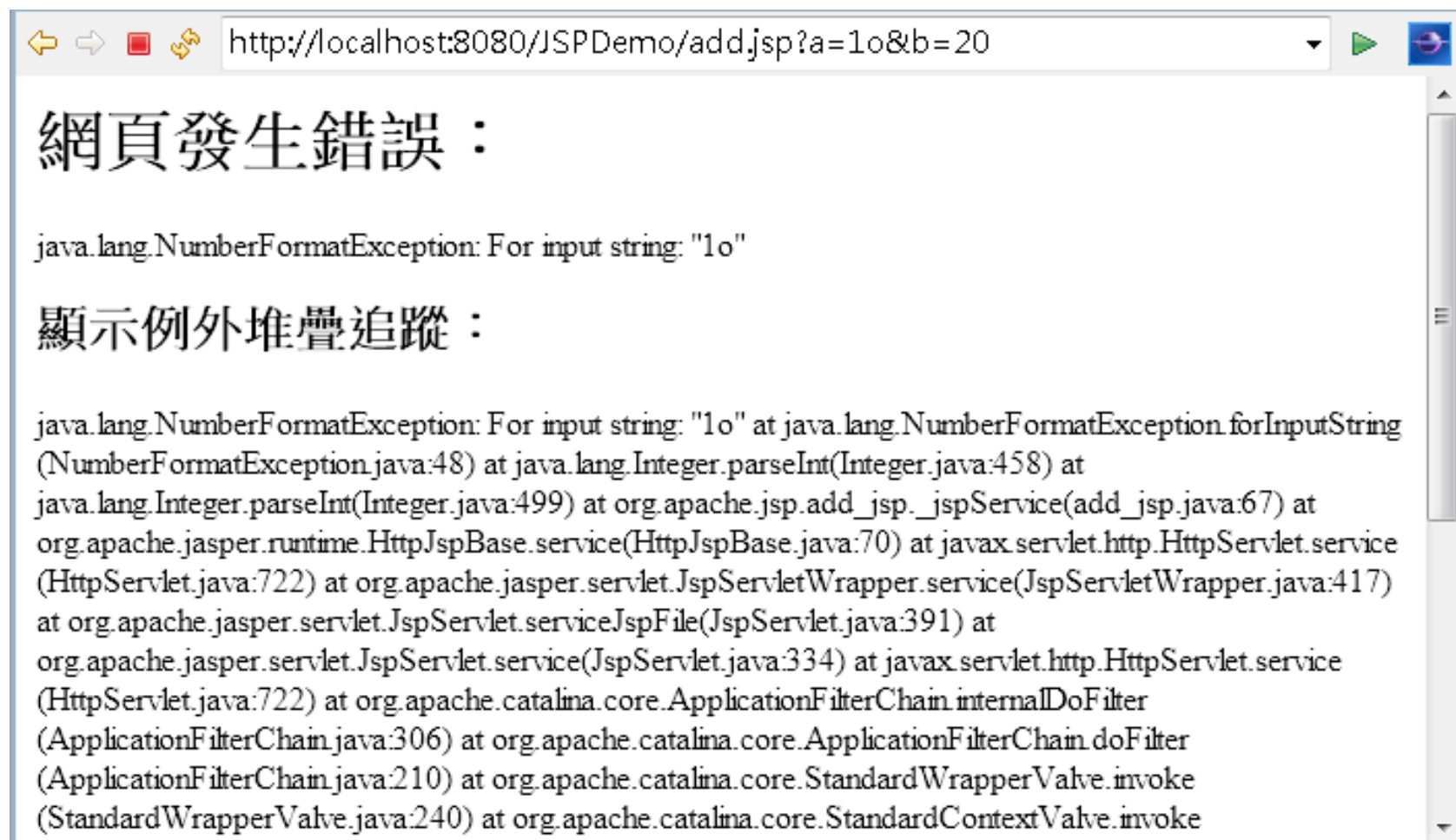
errorPage屬性

```
<%@page contentType="text/html"
    pageEncoding="UTF-8" errorPage="error.jsp"%> ← 設定 errorPage 屬性
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>加法網頁</title>
</head>
<body>
<%
    String a = request.getParameter("a");
    String b = request.getParameter("b");
    out.println("a + b = " +
        (Integer.parseInt(a) + Integer.parseInt(b))
    );
%>
</body>
</html>
```

errorPage屬性

```
<%@page contentType="text/html" pageEncoding="UTF-8"
        isErrorPage="true"%> ← 設定 isErrorPage 屬性
<%@page import="java.io.PrintWriter"%>
<html>
<head><title>錯誤</title></head>
<body>
    <h1>網頁發生錯誤:</h1><%= exception %>
    <h2>顯示例外堆疊追蹤:</h2>
<%
    exception.printStackTrace(new PrintWriter(out));
%>
</body>
</html>
```

errorPage屬性



web.xml設定

```
<web-app ...>
  <error-page>
    <exception-type>java.lang.NullPointerException</exception-type>
    <location>/report.view</location>
  </error-page>
</web-app>
```

```
<web-app ...>
  <error-page>
    <error-code>404</error-code>
    <location>/404.jsp</location>
  </error-page>
</web-app>
```

<jsp:include>、<jsp:forward>標籤

- 執行時期依條件動態地調整包括的JSP頁面

```
<jsp:include page="add.jsp">  
    <jsp:param name="a" value="1" />  
    <jsp:param name="b" value="2" />  
</jsp:include>
```

- 目前頁面會自己生成一個Servlet類別被包括的add.jsp也會獨立生成一個Servlet類別
- 轉譯而成的Servlet會取得RequestDispatcher物件，並執行include()方法

<jsp:include>、<jsp:forward>標籤

- 將請求轉發給另一個JSP頁面作處理

```
<jsp:forward page="add.jsp">  
    <jsp:param name="a" value="1" />  
    <jsp:param name="b" value="2" />  
</jsp:forward>
```

- 目前頁面會生成一個Servlet，而被轉發的add.jsp也是生成一個Servlet
- 轉譯的Servlet會取得RequestDispatcher物件，並執行forward()方法

簡介<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

- 搭配<jsp:useBean>的JavaBean元件
 - 必須實作java.io.Serializable介面
 - 沒有公開（public）的類別變數
 - 具有無參數的建構式
 - 具有公開的設值方法（Setter）與取值方法（Getter）

簡介<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

```
public class User implements Serializable {  
    private String name;  
    private String password;  
  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getPassword() {  
        return password;  
    }  
    public void setPassword(String password) {  
        this.password = password;  
    }  
  
    public boolean isValid() {  
        return "caterpillar".equals(name) && "123456".equals(password);  
    }  
}
```

```
<%@page import="cc.openhome.*"
      contentType="text/html" pageEncoding="UTF-8"%>
```

```
<jsp:useBean id="user" class="cc.openhome.User"/> ← ❶ 使用<jsp:useBean>
```

```
<jsp:setProperty name="user" property="*" /> ← ❷ 使用<jsp:setProperty>
```

```
    // 略...
    <body>
    <%
        if(user.isValid()) { ← ❸ user 名稱是根據<jsp:useBean>上的 id 名稱而來
    %>
        <h1><jsp:getProperty name="user" property="name"/> 登入成功</h1>
    <%
        } ← ❹ 使用<jsp:getProperty>
    %>
        else {
    %>
        <h1>登入失敗</h1>
    <%
        }
    %>
    </body>
</html>
```

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

```
<jsp:useBean id="user" class="cc.openhome.User" />
```

cc.openhome.User user = null; // id="user" 就是產生這邊的 user 參考名稱

```
synchronized (request) {  
    user = (cc.openhome.User) _jspx_page_context.getAttribute(  
        "user", PageContext.PAGE_SCOPE); // 以及屬性名稱  
    if (user == null){  
        user = new cc.openhome.User();  
        _jspx_page_context.setAttribute(  
            "user", user, PageContext.PAGE_SCOPE);  
    }  
}
```

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

```
<jsp:useBean id="user" class="cc.openhome.User" scope="session"/>
```

```
cc.openhome.User user = null;
synchronized (request) {
    user = (cc.openhome.User) _jspx_page_context.getAttribute(
        "user", PageContext.SESSION_SCOPE);
    if (user == null){
        user = new cc.openhome.User();
        _jspx_page_context.setAttribute(
            "user", user, PageContext.SESSION_SCOPE);
    }
}
```

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

```
<jsp:useBean id="user"  
             type="cc.openhome.BaseUser"  
             class="cc.openhome.User"  
             scope="session"/>
```

```
cc.openhome.BaseUser user = null;  
synchronized (request) {  
    user = (cc.openhome.BaseUser) _jspx_page_context.getAttribute(  
        "user", PageContext.SESSION_SCOPE);  
    if (user == null){  
        user = new cc.openhome.User();  
        _jspx_page_context.setAttribute(  
            "user", user, PageContext.SESSION_SCOPE);  
    }  
}
```

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

- 標籤的目的是減少JSP中Script的使用
- 反過來說，如果發現JSP中有Scriptlet，撰寫的是從某個屬性範圍中取得物件
- 思考是否可以用<jsp:useBean>來消除Scriptlet的使用

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

```
<jsp:setProperty name="user" property="password" value="123456" />
```

- 使用PageContext的findAttribute()，從page、request、session、application依序找看看有無name指定的屬性名稱
- 找到的話，再透過反射機制找出JavaBean上的setPassword()方法，呼叫並將value的指定值設定給JavaBean

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

- 將請求參數的值設定給JavaBean的某個屬性

```
<jsp:setProperty name="user" param="password" property="password" />
```

- 由JSP的自省機制來判斷是否有相同的請求參數名稱

```
<jsp:setProperty name="user" property="password" />
```

- 自省機制自動匹配

```
<jsp:setProperty name="user" property="*" />
```

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

- 使用<jsp:useBean>時一併設定屬性值

```
<jsp:useBean id="user" class="cc.openhome.User" scope="session">  
    <jsp:setProperty name="user" property="*" />  
</jsp:useBean>
```

```
cc.openhome.User user = null;  
synchronized (request) {  
    user = (cc.openhome.User) _jspx_page_context.getAttribute(  
        "user", PageContext.SESSION_SCOPE);  
    if (user == null){  
        user = new cc.openhome.User();  
        _jspx_page_context.setAttribute(  
            "user", user, PageContext.SESSION_SCOPE);  
        org.apache.jasper.runtime.JspRuntimeLibrary.introspect(  
            _jspx_page_context.findAttribute("user"), request);  
    }  
}
```

深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

- 底下這個有點不同

```
<jsp:useBean id="user" class="cc.openhome.User" scope="session"/>  
<jsp:setProperty name="user" property="*" />
```

```
cc.openhome.User user = null;  
synchronized (request) {  
    user = (cc.openhome.User) _jspx_page_context.getAttribute(  
        "user", PageContext.SESSION_SCOPE);  
    if (user == null){  
        user = new cc.openhome.User();  
        _jspx_page_context.setAttribute(  
            "user", user, PageContext.SESSION_SCOPE);  
    }  
}  
org.apache.jasper.runtime.JspRuntimeLibrary.introspect(  
    _jspx_page_context.findAttribute("user"), request);
```

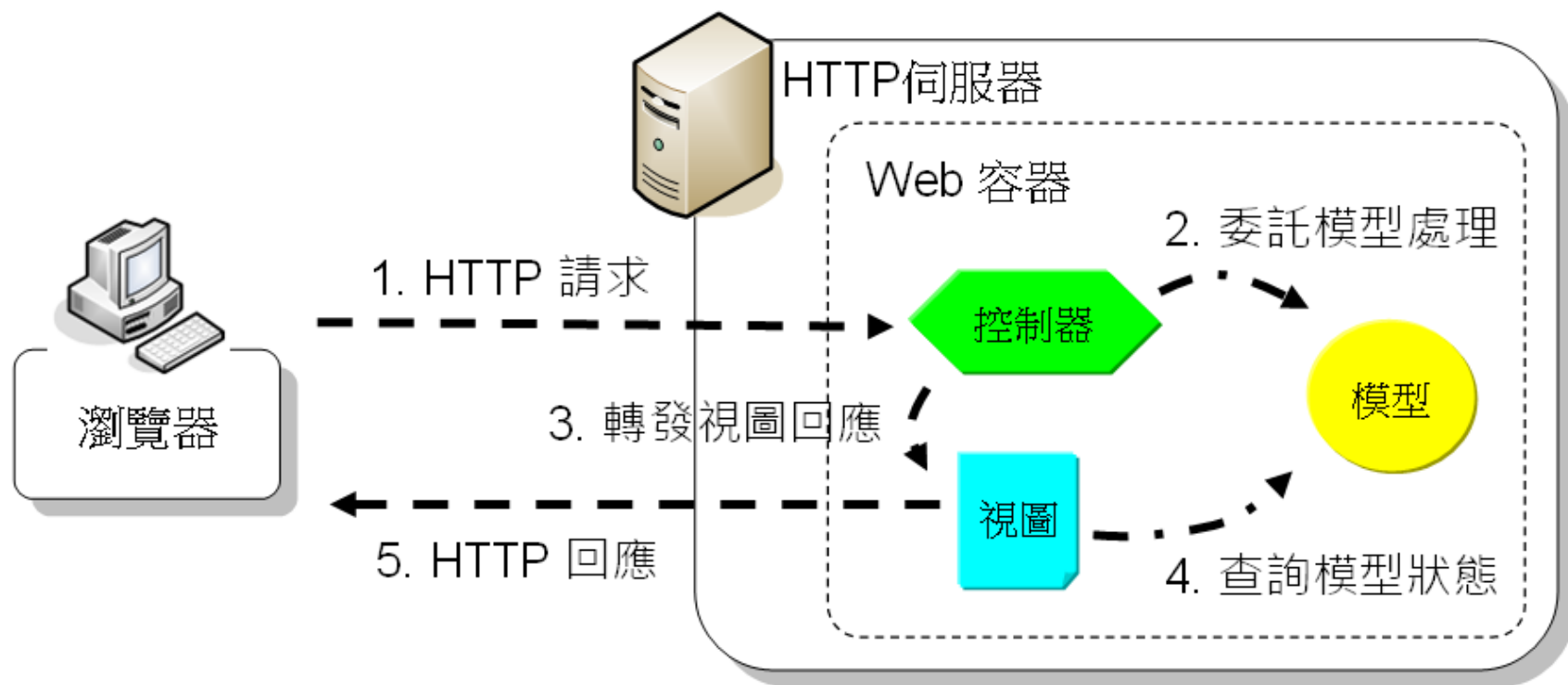
深入<jsp:useBean>、<jsp:setProperty> 與<jsp:getProperty>

- 取得JavaBean屬性

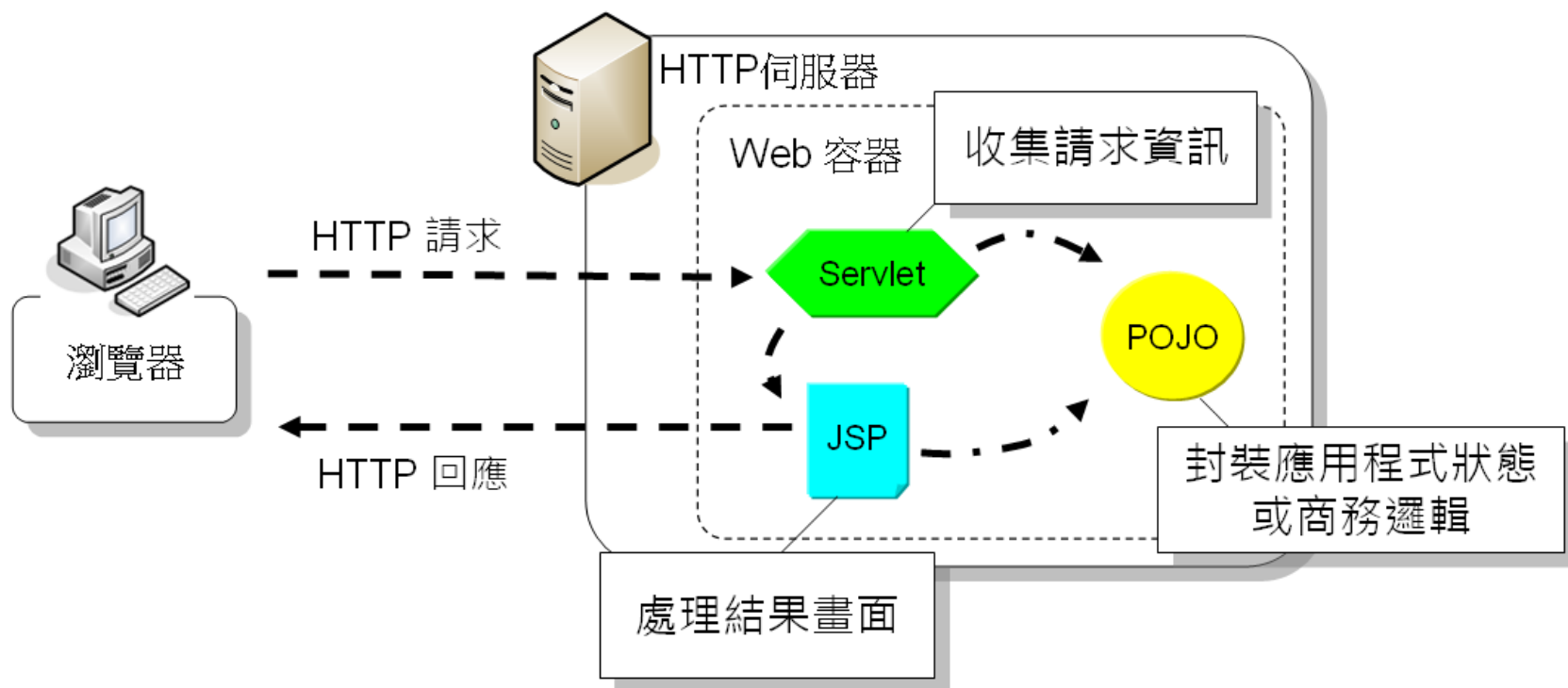
```
<jsp:getProperty name="user" property="name"/>
```

```
out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString(((  
    (cc.openhome.User)_jspx_page_context.findAttribute("user"))  
                                .getName()  
)));
```

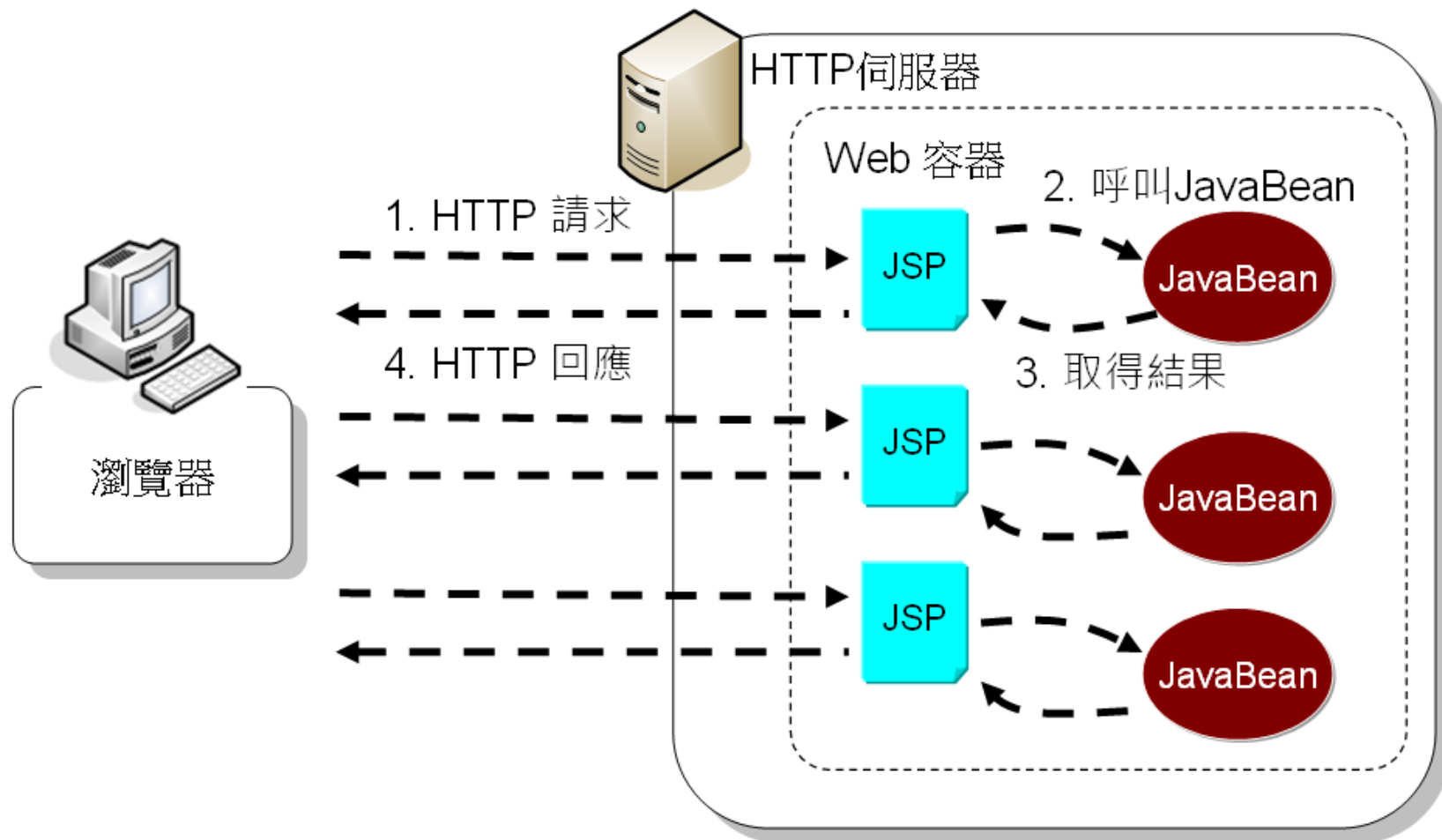
Model 2



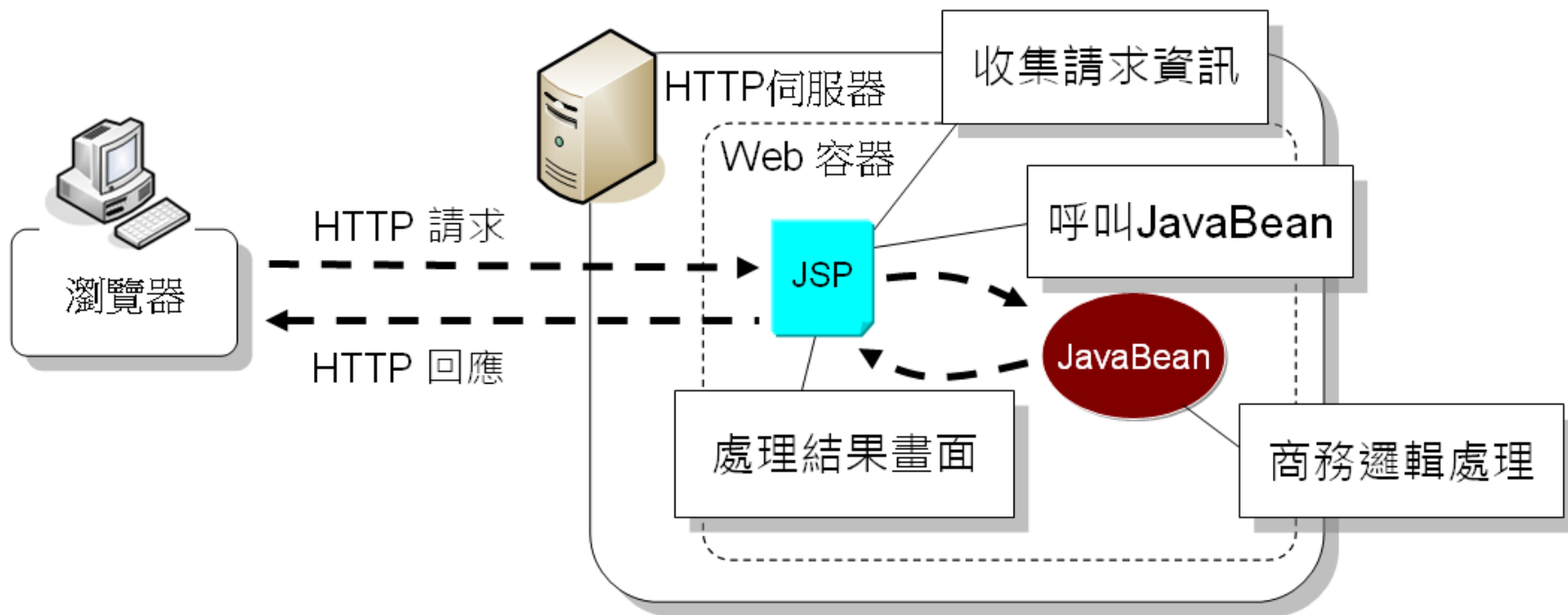
Model 2



談談Model 1



談談Model 1



XML格式標籤

JSP 語法

```
<%@page import="java.util.*" %>
```

```
<%! String name; %>
```

```
<% name = "caterpillar"; %>
```

```
<%= name %>
```

網頁文字

XML 格式語法

```
<jsp:directive.page  
    import="java.util.*"/>
```

```
<jsp:declaration>  
    String name;  
</jsp:declaration>
```

```
<jsp:scriptlet>  
    name = "caterpillar";  
</jsp:scriptlet>
```

```
<jsp:expression>  
    name  
</jsp:expression>
```

```
<jsp:text>
```

網頁文字

```
</jsp:text>
```

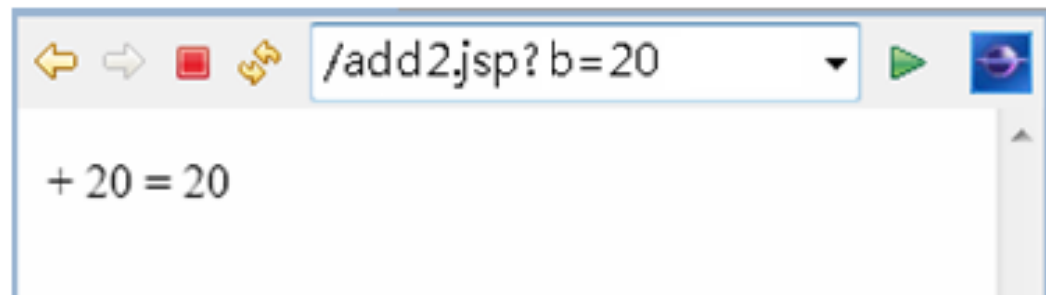
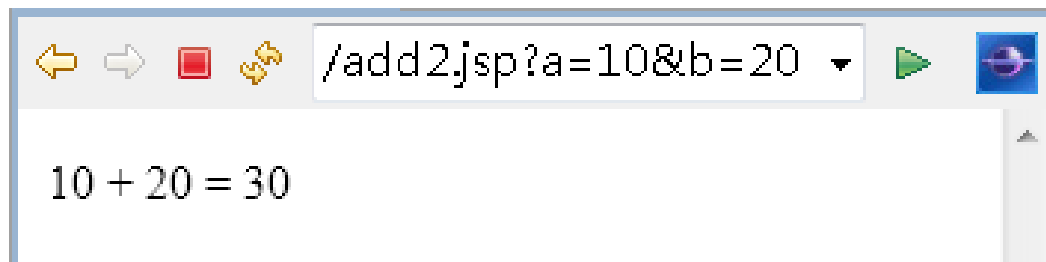
```
<?xml version="1.0" encoding="UTF-8"?>
<jsp:root xmlns:jsp="http://java.sun.com/JSP/Page" version="2.0">
  <jsp:directive.page import="java.util.Date"/>
  <jsp:directive.page contentType="text/html" pageEncoding="UTF-8"/>
  <jsp:element name="text">
    <jsp:body>
<html>
  <head>
    <meta http-equiv="Content-Type"
      content="text/html; charset=UTF-8"/>
    <title>Page 指示元素</title>
  </head>
  <body>
    <h1>現在時間<jsp:expression>new Date()</jsp:expression></h1>
  </body>
</html>
    </jsp:body>
  </jsp:element>
</jsp:root>
```

簡介EL

```
<%@page contentType="text/html" pageEncoding="UTF-8"
    errorPage="error.jsp"%>
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
  <head>
    <meta http-equiv="Content-Type"
        content="text/html; charset=UTF-8">
    <title>加法網頁</title>
  </head>
  <body>
    <%
      String a = request.getParameter("a");
      String b = request.getParameter("b");
      ${param.a} + ${param.b} = ${param.a + param.b} ← 使用 EL
      (Integer.parseInt(a) + Integer.parseInt(b))
    %>
  </body>
</html>
```

簡介EL

- EL優雅地處理了null值的情況，對於null值直接以空字串加以顯示



簡介EL

- EL的點運算子還可以連續存取物件

方法：`<%= ((HttpServletRequest) pageContext.getRequest()).getMethod() %><br>`

參數：`<%= ((HttpServletRequest) pageContext.getRequest()).getQueryString() %><br>`

IP：`<%= ((HttpServletRequest) pageContext.getRequest()).getRemoteAddr() %><br>`

方法：`${pageContext.request.method}<br>`

參數：`${pageContext.request.queryString}<br>`

IP：`${pageContext.request.remoteAddr}<br>`

簡介EL

- 可以使用page指示元素的**isELIgnored**屬性（預設是false），來設定JSP網頁是否使用EL
- 可以在web.xml中設定**<el-ignored>**標籤為true來決定不使用EL

```
<jsp-config>
  <jsp-property-group>
    <url-pattern>*.jsp</url-pattern>
    <el-ignored>true</el-ignored>
  </jsp-property-group>
</jsp-config>
```

簡介EL

- 如果 web.xml 中的 `<el-ignored>` 與 page 指令元素的 `isELIgnored` 設定都沒有設定
 - 2.3 或以下的版本，不會執行 EL
 - 2.4 或以上的版本，則會執行 EL
- 不設定 page 指令元素的 `isELIgnored`
 - web.xml 中的 `<el-ignored>` 為 `false`，則會執行 EL，
 - web.xml 中的 `<el-ignored>` 為 `true`，則不會執行 EL
- 使用 page 指令元素的 `isELIgnored` 設定是否支援 EL，則以 page 指令元素的設定為主

使用EL取得屬性

- 存取屬性

```
<h1><jsp:getProperty name="user" property="name"/>登入成功</h1>
```

```
<h1>${user.name}登入成功</h1>
```

- 存取陣列

```
名稱一: ${array[0]} <br>
```

```
名稱二: ${array[1]} <br>
```

```
名稱三: ${array[2]} <br>
```


使用EL取得屬性

- 如果使用點 (.) 運算子，則左邊可以是JavaBean或Map物件
- 如果使用[]運算子，則左邊可以是JavaBean、Map、陣列或List物件
- 例如JavaBean...

```
${user.name}
```

```
${user["name"]}
```

使用EL取得屬性

- 例如Map...

```
<%  
    Map<String, String> map = new HashMap<String, String>();  
    map.put("user", "caterpillar");  
    map.put("role", "admin");  
    request.setAttribute("login", map);  
%>
```

```
User: ${login.user}<br>  
Role: ${login.role}<br>
```

```
User: ${login["user"]}<br>  
Role: ${login["role"]}<br>
```

使用EL取得屬性

- 當左邊是Map物件時，建議使用[]運算子，因為如果設定Map時的鍵名稱有空白或點字元時，這是可以正確取得值的方式

```
<%  
    Map<String, String> map = new HashMap<String, String>();  
    map.put("user name", "caterpillar");  
    map.put("local.role", "admin");  
    request.setAttribute("login", map);  
%>  
...  
User: ${login["user name"]} <br>  
Role: ${login["local.role"]} <br>
```

使用EL取得屬性

- List的例子

```
<%  
    List<String> names = new ArrayList<String>();  
    names.add("caterpillar");  
    names.add("momor");  
    request.setAttribute("names", names);  
%>  
...  
User 1: ${names[0]}<br>  
User 2: ${names[1]}<br>
```

使用EL取得屬性

- 如果[]運算子中不是使用雙引號，則會嘗試作運算

```
%  
    List<String> names = new ArrayList<String>();  
    names.add("caterpillar");  
    names.add("momor");  
    request.setAttribute("names", names);  
%>  
...  
User : ${names[param.index]}<br>
```

使用EL取得屬性

- [] 中也可以進行巢狀 ...

```
<%  
    List<String> names = new ArrayList<String>();  
    names.add("caterpillar");  
    names.add("momor");  
    request.setAttribute("names", names);  
    Map<String, String> datas = new HashMap<String, String>();  
    datas.put("caterpillar", "caterpillar's data");  
    datas.put("momor", "momor's data");  
    request.setAttribute("datas", datas);  
%>  
// ...  
User data: ${datas[names[param.index]]}<br>
```

EL隱含物件

- `pageContext`隱含物件
- 屬性範圍相關隱含物件
- 請求參數相關隱含物件
- 標頭（Header）相關隱含物件
- `cookie`隱含物件
- 初始參數隱含物件

EL運算子

運算式	結果
<code>\${1}</code>	1
<code>\${1 + 2}</code>	3
<code>\${1.2 + 2.3}</code>	3.5
<code>\${1.2E4 + 1.4}</code>	120001.4
<code>\${-4 - 2}</code>	-6
<code>\${21 * 2}</code>	42
<code>\${3/4}</code> 或 <code>\${3 div 4}</code>	0.75
<code>\${3/0}</code>	Infinity
<code>\${10%4}</code> 或 <code>\${10 mod 4}</code>	2
<code>\${(1==2) ? 3 : 4}</code>	4

EL運算子

運算式	結果
<code>\${true and false}</code>	false
<code>\${true or false}</code>	true
<code>\${not true}</code>	false

EL運算子

運算式	結果
<code>\${1 < 2}</code> 或 <code>\${1 lt 2}</code>	true
<code>\${1 > (4/2)}</code> 或 <code>\${1 gt (4/2)}</code>	false
<code>\${4.0 >= 3}</code> 或 <code>\${4.0 ge 3}</code>	true
<code>\${4 <= 3}</code> 或 <code>\${4 le 3}</code>	false
<code>\${100.0 == 100}</code> 或 <code>\${100.0 eq 100}</code>	true
<code>\${(10*10) != 100}</code> 或 <code>\${(10*10) ne 100}</code>	false
<code>\${'a' < 'b'}</code>	true
<code>\${"hip" > "hit"}</code>	false
<code>\${'4' > 3}</code>	true

自訂EL函式

```
package cc.openhome;

import java.util.Collection;

public class Util {
    public static int length(Collection collection) {
        return collection.size();
    }
}

<%= Util.length(request.getAttribute("someList")) %>

${ util:length(requestScope.someList) }
```

自訂EL函式

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib version="2.1" xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-jsptaglibrary_2_1.xsd">
  <tlib-version>1.0</tlib-version>
  <short-name>openhome</short-name>
  <uri>http://openhome.cc/util</uri> ← 設定 uri 對應名稱
  <function>
    <description>Collection Length</description>
    <name>length</name> ← 自訂的 EL 函式名稱
    <function-class>
      cc.openhome.Util ← 對應的哪個類別
    </function-class>
    <function-signature>
      int length(java.util.Collection) ← 對應至該類別的哪個方法
    </function-signature>
  </function>
</taglib>
```

自訂EL函式

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@taglib prefix="util" uri="http://openhome.cc/util"%> ← 使用 taglib 指示元素
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
  <head>
    <meta http-equiv="Content-Type"
          content="text/html; charset=UTF-8">
    <title>自訂 EL 函式</title>
  </head>
  <body>
    ${ util:length(requestScope.someList) } ← 使用自訂 EL
  </body>
</html>
```

綜合練習 / 微網誌

- 使用JSP改寫先前綜合練習中使用Servlet所實現的視圖網頁